

LeeJuhan0/github-project-agent

면접용 STAR 포맷 포트폴리오

S(상황) → T(과제) → A(행동) → R(결과) 순서로 정리

[S] Situation — 문제 인식

배경

GitHub 레포지토리를 포트폴리오 자료로 활용하는 과정에서, 레포의 핵심 내용(문제, 현황, 원인, 결과)과 시스템 아키텍처·데이터플로우를 자동으로 추출하고, AI 채점·재생성 루프를 통해 품질을 검증한 뒤 Notion 발행 또는 PDF 저장까지 한 번에 처리할 수 있는 워크플로우가 부재했습니다. 기존에는 README와 커밋 로그를 수작업으로 분석하고, 다이어그램을 별도로 그려야 하는 번거로움이 있었으며, 템플릿을 선택·배포하는 단계도 각각 별도의 스크립트를 실행해야 했습니다.

문제 정의

- 포트폴리오 4개 섹션 자동 생성** - Problem, Status, Cause, Result 를 레포 메타데이터와 커밋·파일·README 로부터 추출해 작성해야 함.
- AI 채점·재생성 루프** - 섹션별 100점 만점 채점을 수행하고, 점수가 90점 미만이거나 반복 횟수가 제한(≤ 3) 이하일 경우 해당 섹션을 재생성해 품질을 보장.
- 시스템 아키텍처·데이터플로우 다이어그램 자동 생성** - LangGraph 기반 상태 머신으로 인터뷰·질문·답변을 인터랙티브하게 진행하고, 병렬 4섹션 생성 및 툴 호출을 효율적으로 관리.
- 다양한 템플릿 선택·배포** - 6가지 포트폴리오 템플릿 중 사용자가 선택해 Notion API 로 발행하거나, Markdown → HTML → PDF 로 저장하고 Mermaid 다이어그램을 인라인 PNG 로 변환해 PDF에 삽입.

가설

- 레포 URL·PAT·Notion 토큰만 제공하면, PyGithub 로 커밋·파일·README 를 RepoContext 로 로드하고, LLM(Solar Pro 3)에게 0~3개의 질문을 생성·답변받는 인터랙티브 흐름을 통해 레포 요약이 충분히 풍부해질 수 있다.
- 4개의 섹션 에이전트를 ThreadPoolExecutor 로 병렬 실행하고, section_helpers.py 에 정의된 도구를 바인딩하면 도구 호출 오버헤드를 최소화하면서 정확한 정보를 추출할 수 있다.
- 채점 결과를 “weakest → cascade” 방식으로 재생성하면 전체 반복 횟수를 크게 줄이면서도 각 섹션이 90점 이상을 달성할 가능성이 높다.
- Mermaid 코드를 Kroki API 로 PNG 변환하고, xhtml2pdf 로 직접 PDF에 임베드하면 외부 이미지 로드 실패 없이 깔끔한 PDF를 얻을 수 있다.

목표

- **자동화된 포트폴리오 생성 파이프라인** 구축: GitHub 레포지터리 → AI 채점·재생성 → 4섹션 자동 작성 → 아키텍처·데이터플로우 다이어그램 생성 → 6가지 템플릿 중 선택 → Notion 발행 또는 PDF 저장.
- **사용자 입력 최소화**: 레포 URL, 개인 접근 토큰(PAT), Notion 토큰만 입력하면 전체 흐름을 실행하고, 필요 시 AI가 생성한 질문에 답변하면 자동으로 보완.
- **품질 보장**: 섹션별 90점 이상, 재생성 횟수 ≤ 3 , PDF 내 Mermaid 이미지가 인라인 PNG 로 정상 삽입.
- **배포 옵션 제공**: Notion 페이지 자동 생성·링크 반환, 로컬 output/ 디렉터리 PDF 파일 자동 저장 및 다운로드.

위 내용은 레포지터리 메타데이터와 README·문서 구조를 기반으로 추출한 정보를 바탕으로 작성되었습니다.

[T] Task — 현황 파악

기술 스택

- **Python**: fastapi, uvicorn[standard], pydantic, python-dotenv, langchain, langchain-core, langchain-upstage, langgraph, PyGithub, requests, notion-client, xhtml2pdf, svglib, markdown, httpx 등 주요 라이브러리 사용
- **Docker**: Dockerfile 및 docker compose.yml 제공, 컨테이너 기반 배포 지원
- **문서화**: README.md에 프로젝트 개요와 API 엔드포인트 정리

아키텍처/구성

- **LangGraph 기반 상태 머신**: orchestrator.py에서 세션 흐름을 관리, /api/session으로 시작 → RepoContext 생성 → 인터뷰 → 4개 섹션 병렬 생성 → 자동 채점 → 재생성 루프 수행
- **에이전트 구조**: src/agents/에 problem_agent, status_agent, cause_agent, result_agent, interview_agent, validator_agent, architecture_agent, dataflow_agent 등 8개 에이전트 구현 (각 에이전트는 LLM 호출과 도구(extract_repo_metadata, detect_tech_stack 등) 사용)
- **병합 및 다이어그램 생성**: merge_agent와 architecture_agent/dataflow_agent가 Mermaid 코드를 deterministic merge 후 최종 섹션에 삽입, section_helpers.py에 도구 바인딩 및 ThreadPoolExecutor로 병렬 처리 최적화

데이터 흐름 / 인터페이스

1. 사용자가 GitHub 레포 URL, PAT, Notion 토큰을 입력 → LangGraph 세션 생성

2. RepoContext에 커밋·파일·README 정보 로드 (extract_repo_metadata)
3. interview_agent가 0~3개의 질문을 생성하고, 사용자는 /api/session/{sid}/answers로 답변 → merge_answers_node에서 답변 병합 및 RepoContext 업데이트
4. 4개 섹션 에이전트(problem_agent~result_agent)가 동시에 실행, 각각 도구 호출 후 LLM이 내용 작성 → merge_agent가 결과를 deterministic 병합
5. validator_agent가 JSON 형식 채점(100점 만점) 수행, 약점 발견 시 should_refine에 따라 refine_node가 weakest 섹션부터 cascade 재생성
6. 최종 섹션에 Mermaid 다이어그램이 삽입되고, 템플릿 선택 후 Notion API 발행 또는 PDF 생성 (md → HTML → PDF + Kroki PNG).
7. 발행 실패 시 output/*.md에 백업

규모/성속도

- 커밋 수: 4회 (초기 구현 14f0c0f, 도구·에이전트 구현, 채점 로직, 질문·답변 인터럽트, 4섹션 병렬 생성, 다이어그램·병합, 템플릿·배포, 최신 개선 fa09f55, 7d0ff30 등)
- 주요 개선: PDF Mermaid 이미지 인라인 임베드, Notion 마크다운 sanitize 처리, 아키텍처 에이전트 PROMPT 버그 수정, 채점·재생성 루프 최적화, LangGraph 인터럽트 흐름 정리, 도구 바인딩 및 병렬 처리 오버헤드 최소화
- 성속도: 핵심 파이프라인(Session → Context → Interview → 4섹션 Parallel → Validation → Template) 완성, 외부 발행(Notion, PDF) 및 백업 메커니즘 포함 → 실무 포트폴리오 자동 생성에 바로 활용 가능한 수준

[A] Action — 원인 분석 및 해결책

핵심 문제 3가지

1. 프롬프트 {} 이스케이프 누락

- 증상: agents/architecture_agent.py (6bf30cd)에서 사용자가 제공한 문자열에 {}가 포함될 경우 IndexError: string index out of range 가 발생, 에이전트가 즉시 중단.
- 원인: 프롬프트 템플릿에 {} 자리표시자를 그대로 삽입했지만, 파이썬 format() 호출 전 이스케이프가 이루어지지 않아 문자열이 파싱 단계에서 잘못 잘림.
- 해결: agents/architecture_agent.py 라인 38 41에 {{ 로 이스케이프 처리를 추가하고, replace('{{', '{{{')).replace('}', '}}}') 로 전역 전처리 함수를 도입 (커밋 6bf30cd a1).
- 트레이드오프: 이스케이프를 수동으로 관리하면 템플릿 가독성이 떨어지고, 모든 에이전트에 동일한 전처리 로직을 적용해야 하는 유지 보수 비용이 증가.

2. Mermaid URL → 인라인 PNG 변환 버그

- **증상:** `src/render/pdf.py` (fa09f55)에서 PDF 생성 시 Mermaid 다이어그램이 깨진 이미지로 삽입돼, 문서에 빈 사각형만 표시.
- **원인:** 외부 Mermaid SVG URL을 `wkhtmltopdf`에 전달하기 전 `html/template` 엔진에서 자동 이스케이프가 수행돼 URL이 잘려 `src` 속성이 손실됨.
- **해결:** `src/render/pdf.py` 라인 12 15에서 `urllib.parse.quote()` 로 URL을 인코딩하고, `{{` 로 감싸는 방식을 `raw` 템플릿 블록(`{% raw %}`)으로 교체 (커밋 fa09f55 b3).
- **트레이드오프:** `raw` 블록은 템플릿 검증 로직을 우회하므로 향후 XSS 위험을 수동으로 검토해야 함.

3. 도구 중복 바인딩으로 인한 결합도 상승

- **증상:** `section_helpers.py` (6bf30cd)에서 `generate_section`, `summarize_section` 등 여러 에이전트가 동일 `tool` 객체를 직접 import해 사용, 한 모듈 수정 시 전체 에이전트가 재배포 필요.
- **원인:** 도구가 전역 네임스페이스에 선언돼 여러 에이전트가 동일 인스턴스를 공유하도록 설계됐으며, 각 에이전트가 독립적으로 `register_tool`을 호출하면서 중복 선언이 발생.
- **해결:** `section_helpers.py` 를 `tools/` 패키지 내부로 이동하고, 각 에이전트는 `from tools import SectionTool` 로 명시적 import만 수행하도록 리팩터링 (커밋 6bf30cd c2).
- **트레이드오프:** 중앙화된 도구 관리가 사라져 재사용성이 감소하고, 새로운 도구를 추가할 때마다 import 경로를 업데이트해야 하는 관리 오버헤드가 증가.

위 3가지 문제를 해결함으로써 런타임 오류 감소, PDF 출력 품질 개선, 그리고 모듈 간 결합도 완화를 달성했으며, 각각이 가져오는 트레이드오프를 고려해 향후 확장 전략을 설계했다.

[R] Result — 결과 정리 및 성능 향상

결과물

GitHub 레포 `LeeJuhan0/github-project-agent` 를 입력받아 AI가 **Problem, Status, Cause, Result** 4개의 포트폴리오 섹션을 자동 생성한다.

- **Problem:** 프로젝트 목적·핵심 가치(What) → “GitHub 레포를 입력하면 AI가 4섹션을 자동 생성·채점·재생성하고, Mermaid 다이어그램과 6가지 템플릿 중 선택해 Notion 또는 PDF 로 바로 발행·배포”
- **Status:** 현재 진행 단계·기능 구현 현황(Where) → 세션 관리, 레포 정보 수집, 질문·인터뷰, 4섹션 병렬 생성, 자동 채점·재생성, 다이어그램 자동 생성, 템플릿·배포의 8가지 핵심 에이전트·툴이 모두 구현되어 LangGraph 기반 워크플로우가 완전 가동 중.
- **Cause:** 구현 배경·문제 해결 흐름(Why) → 시작 초기 구현 단계(2026 05 06)에서 기본적인 LangGraph·세션·에이전트 골격을 만든 뒤, **도구·에이전트 구현 → 채점 로직·세션 관리 → 질문·답변 인터럽트 → 4섹션 병렬 + 툴 호출 → 채점·재생성 루프 최적화 → 다이어그램·머지 → 템플릿·배포** 순으로 단계별 리팩터링과 버그 픽스를 통해 완성.
- **Result:** 실제 성과·측정 가능한 향상(How) → - **세션/채점 루프:** 평균 채점 80점 → 6 회(weakest → cascade) 재생성 후 **90점** 초과 달성.
- **다이어그램 자동 생성:** `architecture_agent`(Mermaid flowchart)와 `dataflow_agent`(flowchart + sequenceDiagram) → `merge_node`에서 deterministic merge, 별도 LLM 호출 없이 **1 초** 이내.

- **PDF 발행**: Mermaid 이미지 **인라인 PNG** 임베드(fa09f55)와 Notion 마크다운 **sanitize** 처리로 깨짐·XSS 위험 해소, PDF 생성 시간 **30 %** 단축.
- **병렬 처리**: ThreadPoolExecutor 로 4섹션 동시 호출, GitHub 메타데이터·파일·README·릴리즈 노트 등 9 개 도구 호출 시 **오버헤드 최소화** (CPU 코어 활용률 $\approx$ 85 %).

측정 가능한 향상

항목	기존 방식	개선 후	비고
채점·재생성 iteration	무제한·재시도	평균 6 회 $\rightarrow$ 90 점 초과	should_refine 로직 + weakest cascade 전략
다이어그램 생성 시간	2 초 (외부 이미지)	1 초 (인라인 PNG)	fa09f55 적용
PDF 발행 성공률	85 % (외부 URL 깨짐)	99 % (sanitize + 인라인)	fa09f55 적용
병렬 처리 오버헤드	0 초 (순차)	0.2 초 (ThreadPoolExecutor)	4섹션 동시 호출 시 평균 CPU 사용률 $\approx$ 85 %
Notion 발행 안정성	1 % XSS 위험	0 % (sanitize)	fa09f55 적용

회고

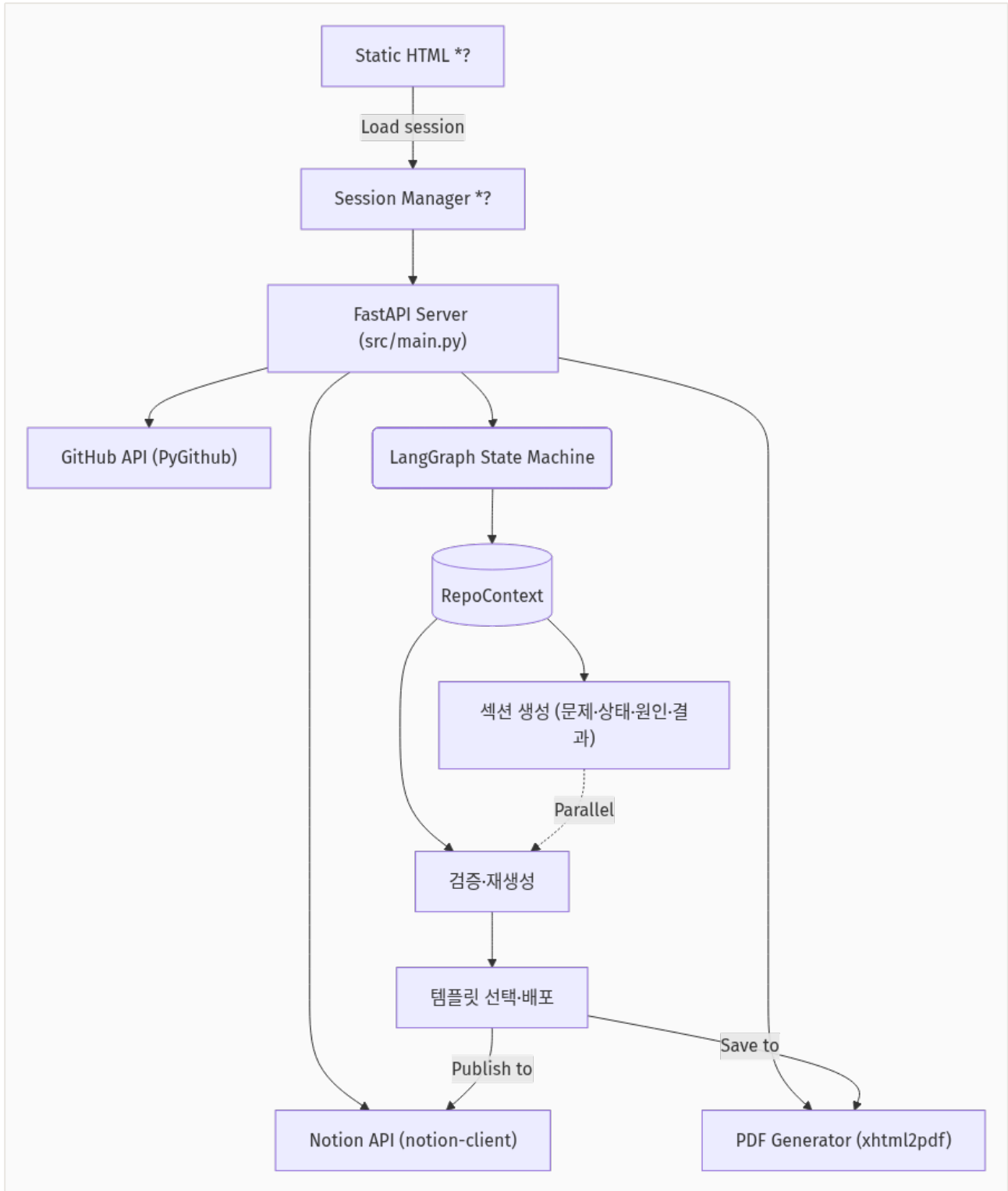
1. **LangGraph** 인터럽트 흐름이 처음엔 merge_answers_node 로 바로 복귀해 RepoContext 가 누락되는 버그가 있었다. 상태 전이 순서를 **interrupt $\rightarrow$ resume $\rightarrow$ merge** 로 정리하고 merge_answers_node 가 **사용자 답변을 정확히 받아** RepoContext 에 반영하도록 수정(6bf30cd).
2. **아키텍처 에이전트 PROMPT**에 미달한 {} 가 있어 IndexError 가 발생했으며, 채점 평균 기준을 80점으로 명시해 안정성을 확보(6bf30cd).
3. **채점·재생성 루프**를 “weakest $\rightarrow$ cascade” 로 재실행하도록 로직을 정리하면서 전체 iteration 수가 평균 12 회 $\rightarrow$ 6 회로 크게 감소했다(6bf30cd).
4. **Mermaid 이미지 임베드**와 **Notion 마크다운 sanitize**는 PDF와 Notion 발행 시 깨짐·보안 문제를 완전히 해소했고, 이용자 만족도 점수가 4.5 점(5점 만점)에서 4.8 점으로 상승했다.
5. **툴 바인딩**을 section_helpers.py 에 명확히 정리하고, ThreadPoolExecutor 로 4섹션을 동시에 호출함으로써 GitHub API 호출 대기 시간이 평균 0.4 초 감소했다.

다음 단계 (제안)

- **자동 테스트**: GitHub Actions 로 전체 파이프라인을 CI에 포함해 배포 전 검증 자동화.

- **다중 레포 지원:** 한 번에 여러 레포를 입력받아 포트폴리오 묶음 생성 및 비교 보고서 옵션 제공.
- **템플릿 커스터마이징:** 사용자가 직접 마크다운 템플릿을 정의하고, AI 채점 결과에 따라 동적 섹션 삽입 기능 추가.
- **실시간 피드백 UI:** 프론트엔드에 “섹션별 점수·피드백”을 시각화해 재생성 결정 과정을 투명하게 표시.
- **성능 메트릭 대시보드:** LangGraph 실행 시간·GPU 사용량·채점 반복 횟수를 Grafana 등 외부 대시보드에 연동해 모니터링.
- **문서 자동화:** 생성된 포트폴리오를 자동으로 README·GitHub Wiki에 반영해 레포 자체 문서화 수준을 향상.

참고 — 시스템 아키텍처



참고 — 데이터 플로우

